



UNIVERSITY OF SCIENCE  
HO CHI MINH CITY

# Introduction to Test Automation

Nguyen V. Vu

# Topics covered

- Overview of test automation
- Automated unit testing
- Typical test automation process
- Testing approaches
- Automation tools

# Test Automation

- Test automation refers to the use of computers to perform testing activities automatically
- Automated testing tools can enter data, run tests, compare results, and report test results
- Automated testing vs. manual testing
  - ❑ Manual testing: tests are performed by humans
  - ❑ Automated testing: tests are performed by computers

# Why Test Automation?

- Manual testing is time and cost consuming
- Automation testing shorten tests and project duration
- Difficult to do manual testing in some situations
  - Multi lingual sites
  - Performance test
  - Security test
- Automation helps increase test coverage
- Manual testing can become tedious and error prone

# Benefits of Test Automation - 1

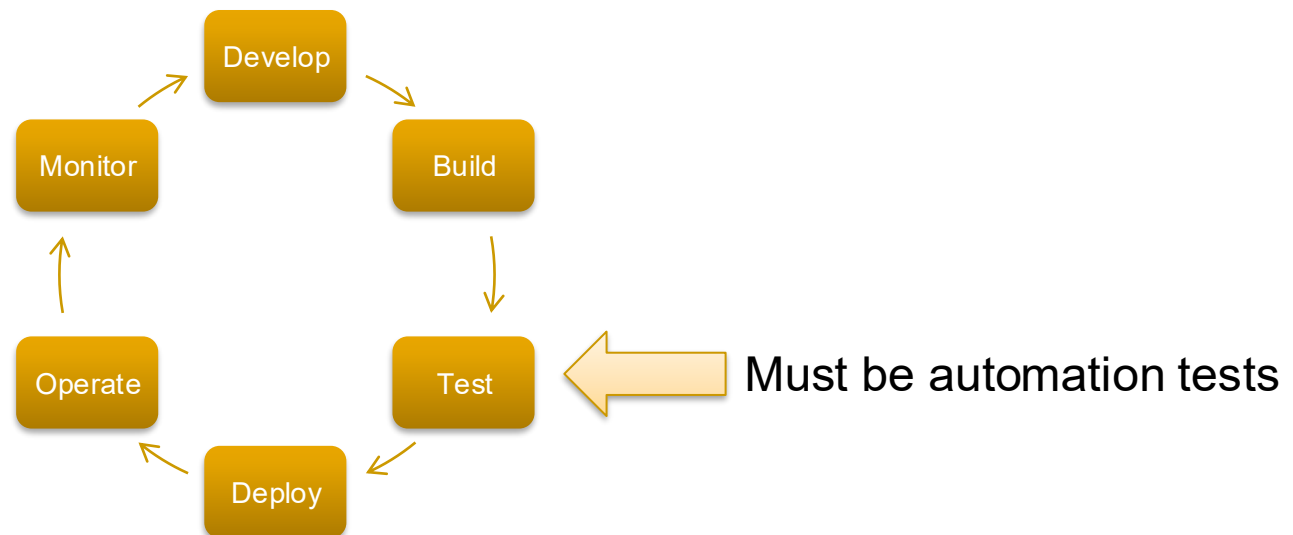
- Saves Time and Cost
  - Higher ROI (Return on Investment)
- Faster than the manual testing
  - Which one?
- Early time to market
  - Better speed in executing tests
- Reusable testing
  - Which one?

# Benefits of Test Automation - 2

- Wider test coverage of application features
  - Test coverage?
  - Why?
- Reliable in results
  - Why?
- Improves accuracy
- Test more frequently and thoroughly
  - Why?

# Test automation is essential for DevOps

- DevOps – a practice for making steps from Development to Operation easy and quick
- DevOps reduces software lifecycle time significantly
- It becomes a trend today



# When Test Automation Works Best?

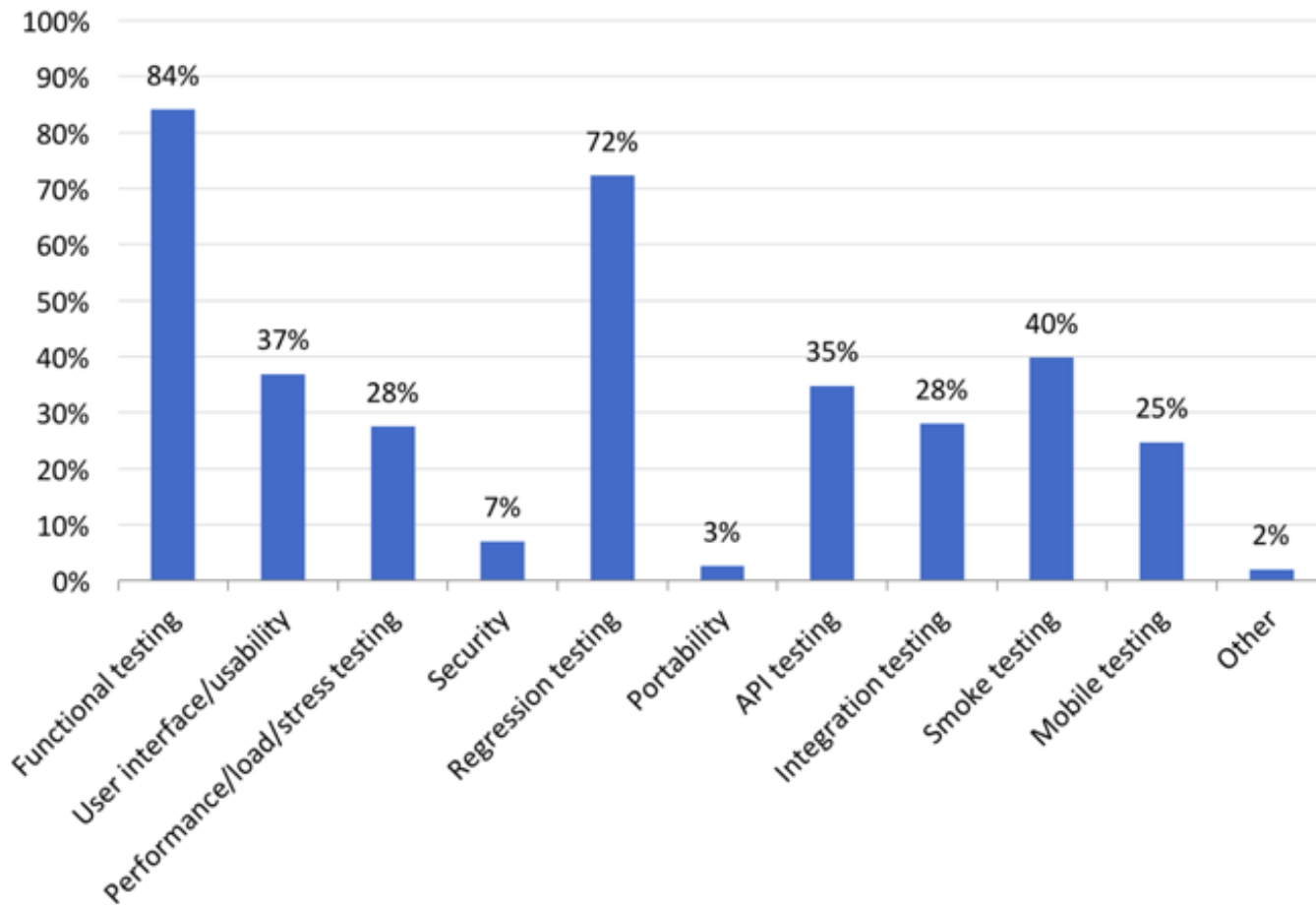
- Test automation works best for some testing
  - ❑ High risk, business critical test cases
  - ❑ Test cases that are executed repeatedly
    - Such as regression test
  - ❑ Test cases that are very tedious or difficult to perform manually
  - ❑ Time consuming test cases
  - ❑ Performance tests
  - ❑ Load tests
  - ❑ Security tests



# When Test Automation Works Best?

- Generally, REGRESSION testing
- Regression testing
  - Testing functionality that has been tested before in earlier iterations

# Testing types using automation

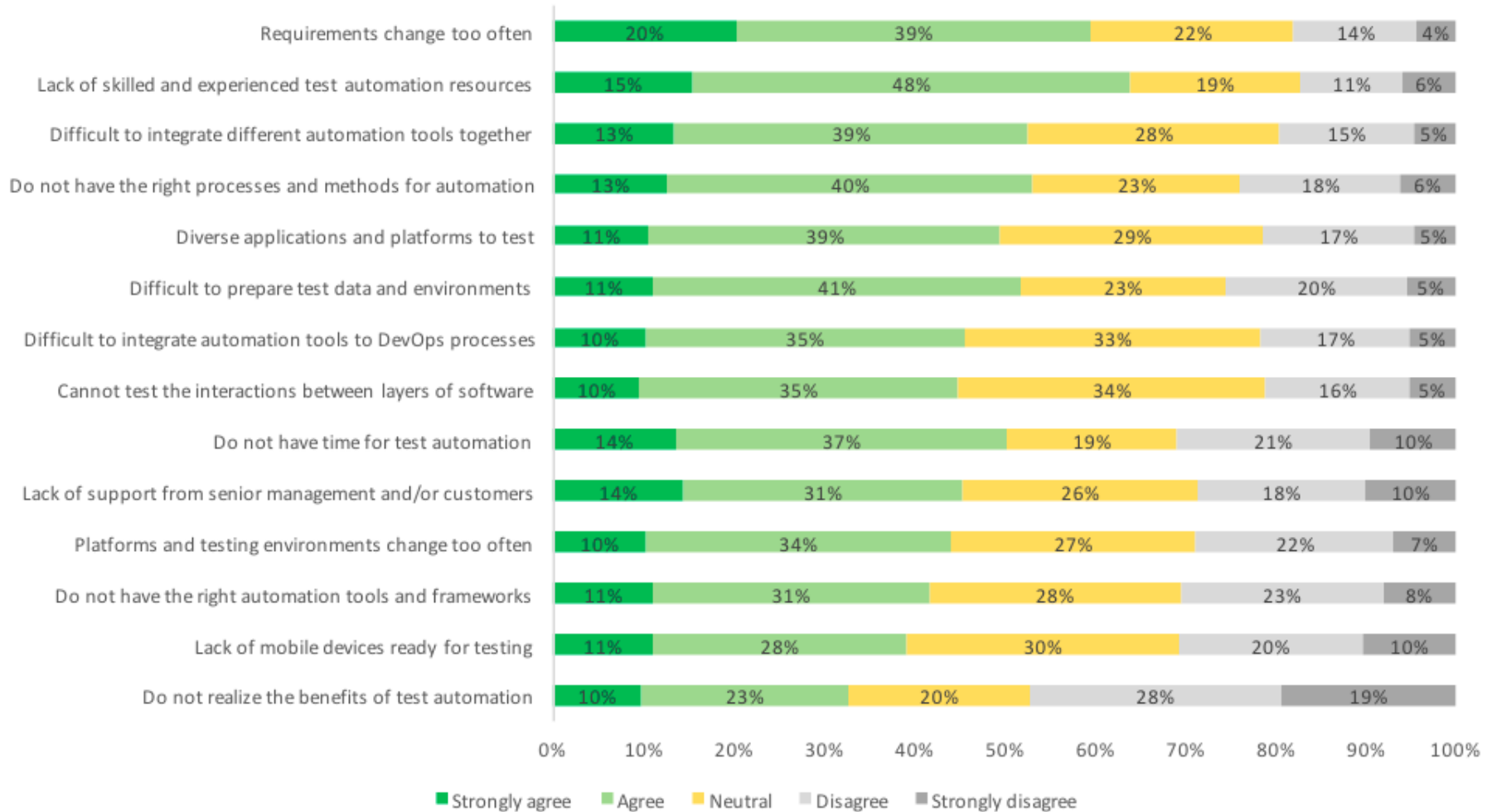


Source: “The most striking problems in test automation: A survey”, 2018. Katalon.com

# When Test Automation Not Suitable?

- Newly designed test cases
  - Test cases should be manually tested at least once
- Test cases for frequently changed requirements
- Ad-hoc test cases
- UI/UX testing
  - Dependent on human judgment and experience

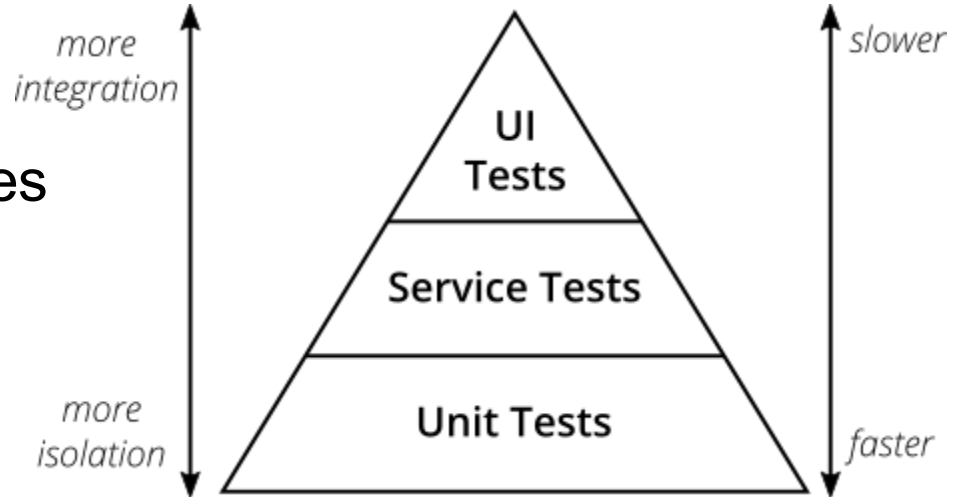
# Challenges of test automation



Source: “The most striking problems in test automation: A survey”, 2018. Katalon.com

# Levels of test automation

- Unit testing
  - Methods, functions, classes
- Integration testing
  - Integration multiple parts
- System testing
  - Focusing on UI, user's features



The test Pyramid

Source: [martinfowler.com](http://martinfowler.com)

# Topics covered

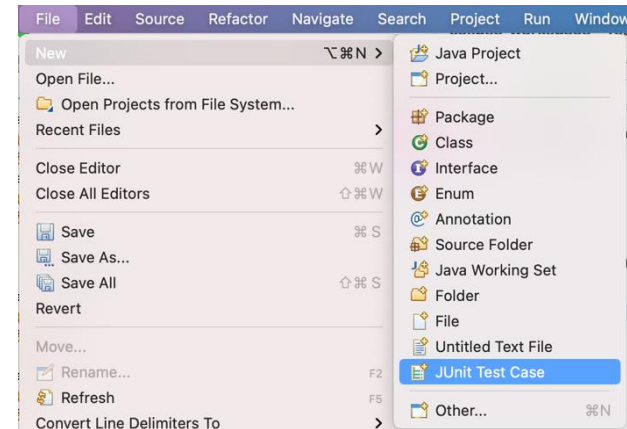
- Overview of test automation
- ➔ ■ Automated unit testing
- Typical test automation process
- Testing approaches
- Automation tools

# JUnit

- JUnit is a framework for unit testing of Java code
- JUnit is used to test Java methods and classes
- Current version: Junit 6
- Test class
  - ❑ Used for testing purpose
  - ❑ Should name \*Test (e.g., RectangleTest)
- Test method
  - ❑ One or more methods defined within a test class
  - ❑ Use @Test annotation
  - ❑ The name should start with “test\*”

# Create a test class in Eclipse

- By default, it is already included in Eclipse
  - Otherwise, download and install
- Create a test class by choosing JUnit Test Case





# A simple test class

```
import org.junit.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import fuv.edu.Rectangle;

public class RectangleTest {

    @Test
    public void testGetArea() {
        Rectangle rec = new Rectangle (10, 20);
        assertEquals(10 * 20, rec.getArea(), "Succeed");
        rec.setWidth(5);
        assertEquals(5 * 20, rec.getArea(), "Succeed");
        assertEquals(10 * 20, rec.getArea(), "Succeed");
    }
}
```

# A more complete test class

```
import org.junit.Test;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import static org.junit.jupiter.api.Assertions.assertEquals;
import java.util.Random;
import fuv.edu.Rectangle;
```

```
public class RectangleTest {
    @BeforeEach
    void setUp () {
    }

    @Test
    @DisplayName("Test the getArea method of Rectangle")
    public void testGetArea() {
        Rectangle rec = new Rectangle(10, 20);
        assertEquals(10 * 20, rec.getArea(), "getArea works with 10 x 20");
    }

    @RepeatedTest(5)
    public void testGetArea2 () {
        Random rand = new Random();
        int width = rand.nextInt(1000);
        int height = rand.nextInt(1000);
        Rectangle rec = new Rectangle(width, height);
        assertEquals(width * height, rec.getArea());
    }
}
```

Before each test run

Indicates a test method

Name of test case shown to users

Test assertion

Repeat this test 5 times

# Assert statements

- Assert statements are used to verify whether the test runs successfully or not
- Common assert statements

| Assertions                   | Notes                                                                            |
|------------------------------|----------------------------------------------------------------------------------|
| assertEquals                 | Check whether the actual value equals the expected. Many assertEquals statements |
| assertArrayEquals            | Check whether two arrays are equal or not                                        |
| assertTrue and assertFalse   | Check the actual value is true or false, respectively                            |
| assertNull and assertNotNull | Check whether a variable is null or not null, respectively                       |
| assertSame                   | Asserts that expected and actual refer to the same object                        |
| assertThrows                 | Asserts that execution of the supplied executable throws an exception            |
| assertAll                    | Asserts that all supplied executables do not throw exceptions.                   |

Other assertions: <https://junit.org/junit5/docs/5.0.1/api/org/junit/jupiter/api/Assertions.html>

# Test for exceptions

- Test whether the tested method throws an exception

@Test

```
void testSetAge() {  
    Employee emp = new Employee();  
    assertThrows(IllegalArgumentException.class,  
emp.setAge(-1));  
}
```

# Assert All Tests

- In a test (or a test method), if one assertion fails, the test stops
- Use `assertAll` to make sure all asserts are checked

```
@Test
void testGroupAsserts() {
    Rectangle rec = new Rectangle();
    assertAll(assertEquals(10 * 20, rec.getArea(),
                          assertThrows(IllegalArgumentException.class, rec.setWidth(-5))
    )
}
```

# Tools for automated unit testing

## ■ Open source

- ❑ JUnit – Java
- ❑ TestNG – Java
- ❑ Pytest – Python
- ❑ Jest – JavaScript
- ❑ Vitest – JavaScript

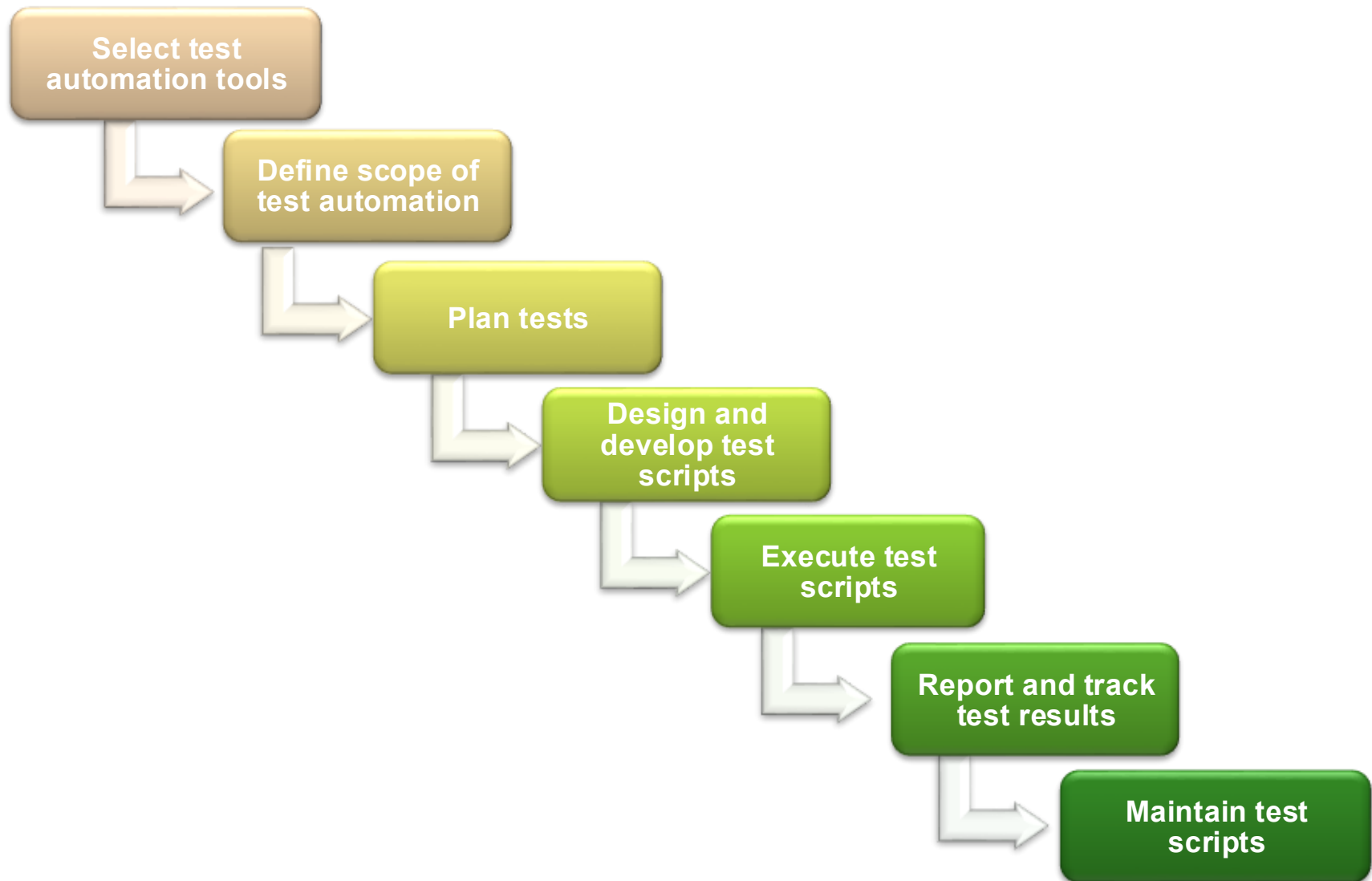
## ■ Commercial tools

- ❑ Diffblue
- ❑ Parasoft Jtest
- ❑ Github Copilot
- ❑ Claude Code

# Topics covered

- Overview of test automation
- Automated unit testing
- ➔ ■ Typical test automation process
- Testing approaches
- Automation tools

# Typical Test Automation Process

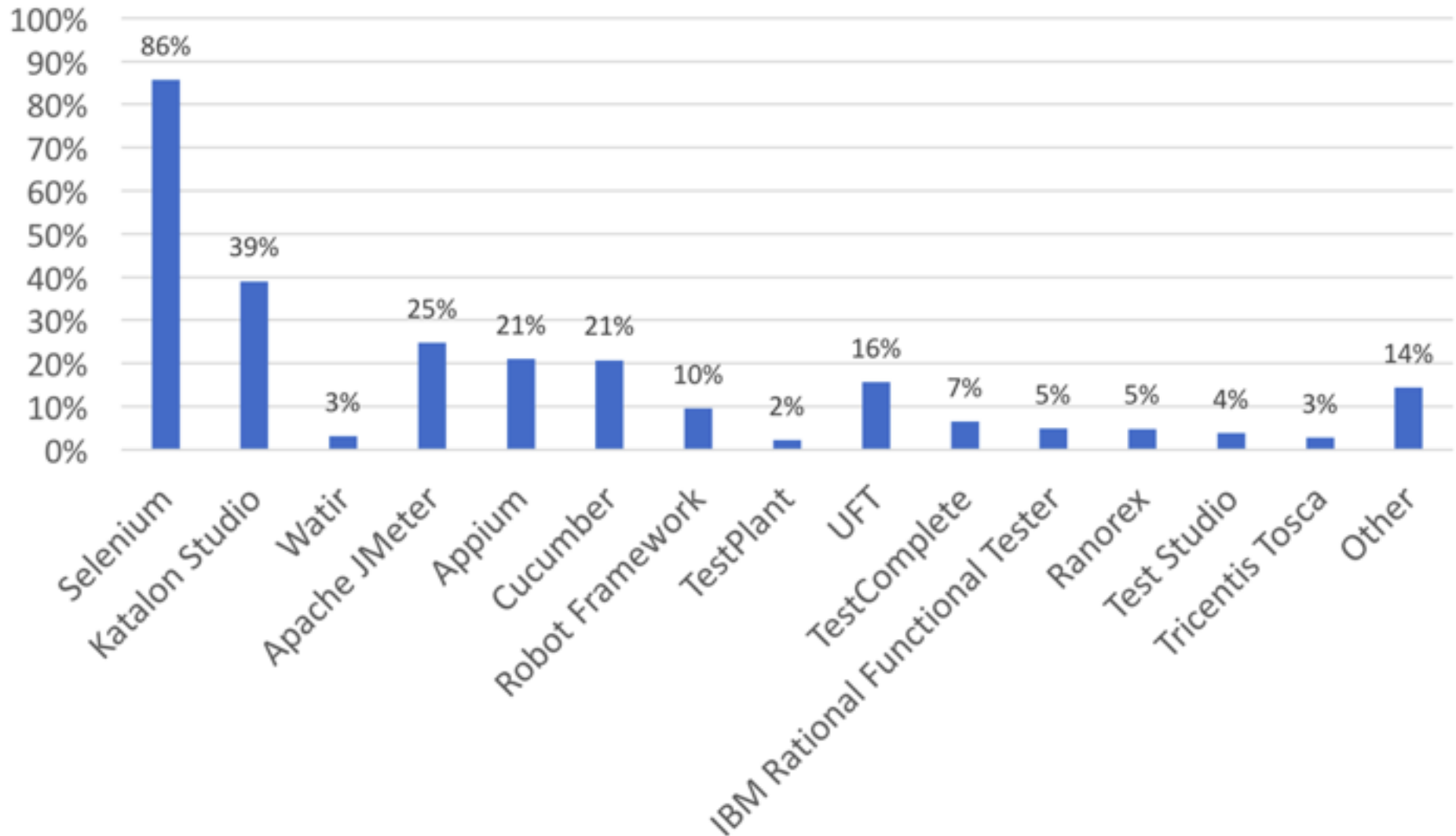




# Select Automation Tools

- Selecting tools suitable for applications under test (AUT) is very challenging
- Types of tool
  - ❑ Commercial: some are powerful but expensive
  - ❑ Open source: free but limited functionality
- Criteria to evaluate
  - ❑ Budget
  - ❑ Easy of use
  - ❑ Scripting languages
  - ❑ Platform (Windows, Unix, Mac OS, etc)
  - ❑ Training
  - ❑ Team experience

# Most popular tools



Source: “The most striking problems in test automation: A survey”, 2018. Katalon.com

# Automation Tool Evaluation

| Criteria                     | <u>Maveryx</u>                                                     | <u>Selenium</u>                               | <u>Cucumber</u>                            | <u>TestComplete</u>                                  | <u>Ranorex</u>                                | <u>UFT (QTP)</u>                              |
|------------------------------|--------------------------------------------------------------------|-----------------------------------------------|--------------------------------------------|------------------------------------------------------|-----------------------------------------------|-----------------------------------------------|
| Open source                  | Yes                                                                | Yes                                           | Yes                                        | No                                                   | No                                            | No                                            |
| Platform                     | Windows, Linux and Mac                                             | Cross-platform                                | Cross-platform                             | Windows (mainly), Android, iOS                       | Windows (mainly), Android, iOS                | Windows (mainly)                              |
| AUT programming languages    | Java                                                               | Web-based languages                           | Ruby, Java, .NET, Flex or web applications | Many                                                 | Many                                          | Many                                          |
| Scripting languages          | Java                                                               | Many (Java, C#, Perl, Python, etc.)           | Ruby, Java, C#                             | VBScript, Jscript, DelphiScript, C++Script, C#Script | C#, VB.NET                                    | VBScript                                      |
| Support                      | 7 (very active)                                                    | 9                                             | 7                                          | 9                                                    | 8                                             | 9                                             |
| Usability                    | 8 (easy to use)                                                    | 7                                             | 6                                          | 9                                                    | 8                                             | 9                                             |
| Script maintainability       | 6 (smart object detection)                                         | 3 (tool provides little support)              | 1 (almost no support by tool)              | 7                                                    | 6                                             | 7 (smart object detection and correction)     |
| Required programming skills  | 7 (high level)                                                     | 6 (support with record/playback)              | 5                                          | 7 (strong record/playback capabilities)              | 6                                             | 8                                             |
| Automated testing approaches | Data-driven, keyword-driven                                        | Record/ playback, keyword-driven, data-driven | Structured, keyword-driven, data-driven    | Record/ playback, keyword-driven, data-driven        | Record/ playback, keyword-driven, data-driven | Record/ playback, keyword-driven, data-driven |
| Cost                         | Free (community edition), from €940/license (professional edition) | Free                                          | Free                                       | From \$999/license                                   | From €609/ license, one year maintenance      | From \$8000/ license                          |

Source: <https://www.katalon.com/resources-center/blog/a-structured-evaluation-for-selecting-a-right-automated-testing-tool/>



# Define Scope of Test Automation

- Which areas in AUT to be automated
- Which areas to be tested manually
  
- Areas to be considered
  - ❑ Business important features
  - ❑ Scenarios which have large amount of data
  - ❑ Common functionalities across applications
  - ❑ Reused business components
  - ❑ Complexity of test cases
  - ❑ Test cases for cross browser testing

# Plan tests

- Define test plans and strategies for testing
  - Tools to be used
  - Test approaches
    - Functional, non-functional, usability, performance, security, testing, ...
    - Automation testing approaches
    - Unit testing, system testing, integration testing, acceptance testing
  - Schedule and timeframe
  - Staffing
  - Strategies for testing
    - Manual testing vs. automation testing, areas to test, test environments, etc.

# Design and Develop Test Scripts

- Design test cases
  - Design test data
  - Design and develop test framework and test scripts
  - Evaluate test scripts
- 
- Similar to programming
- 
- Test framework: a set of rules for automation
  - Test framework consists of software systems, function libraries, reusable modules, data source, etc.

# Execute Tests, Report and Track Test Results

- Run test scripts on AUT
- Test execution is usually controlled by the automation tool
- Test results are compared with expected results
- Test results are reported
- Defects are captured and entered to defect management systems

# Maintain Test Scripts

- Test scripts have to be maintained often as software changes
  - New functions are added, updated
  - Requirements are changed
- As developers change source code, test scripts may not work anymore
  - Parameters are changed
  - GUI objects are changed
  - Outputs are changed
- Maintaining test scripts can be very time consuming



# Topics covered

- Overview of test automation
- Typical test automaion process
- ➔ ■ Automation testing approaches
- Automation tools

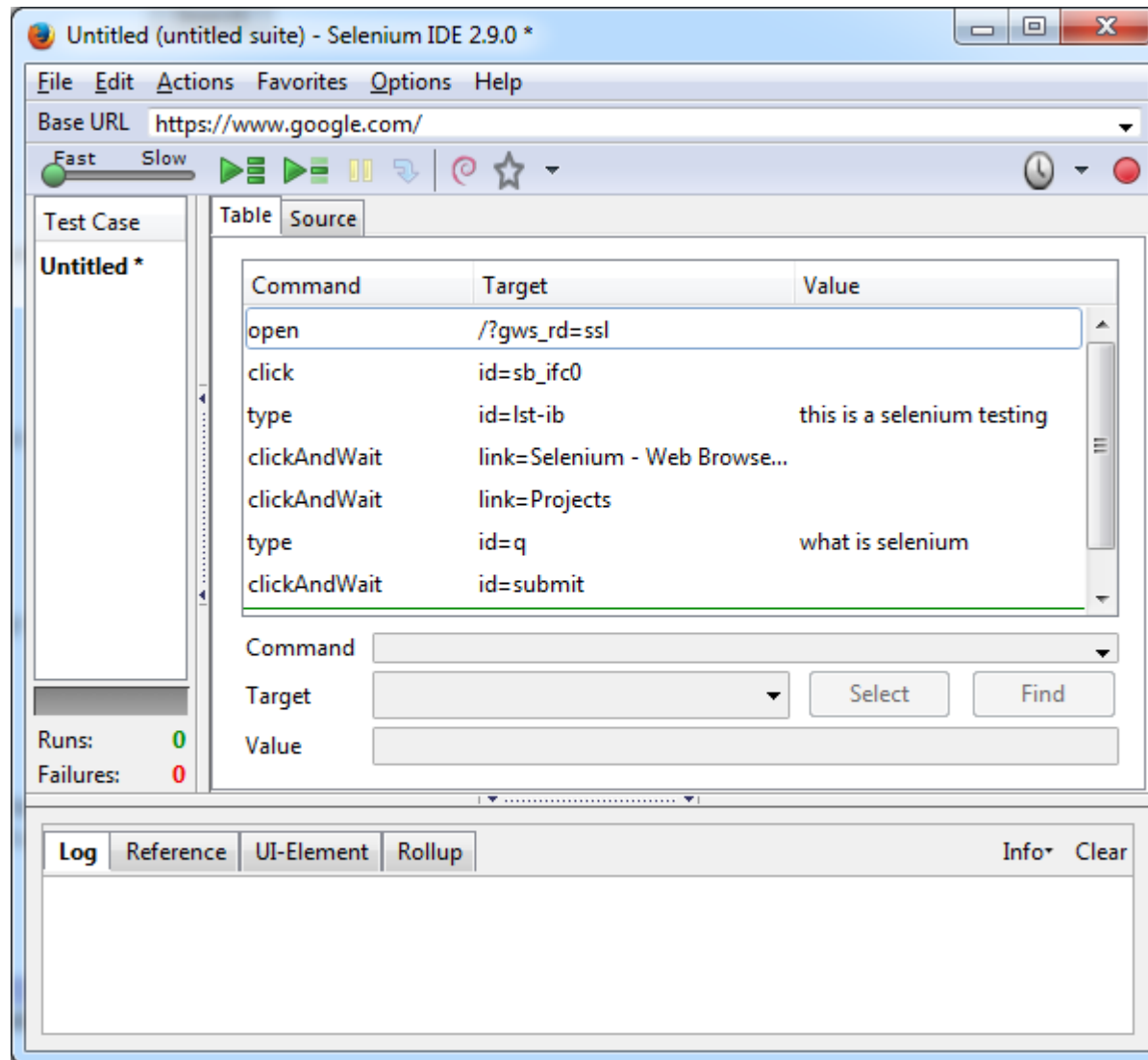
# Scripting Approaches

- Record and playback
- Linear scripting
- Modular scripting
- Data-driven testing
- Keyword-driven testing

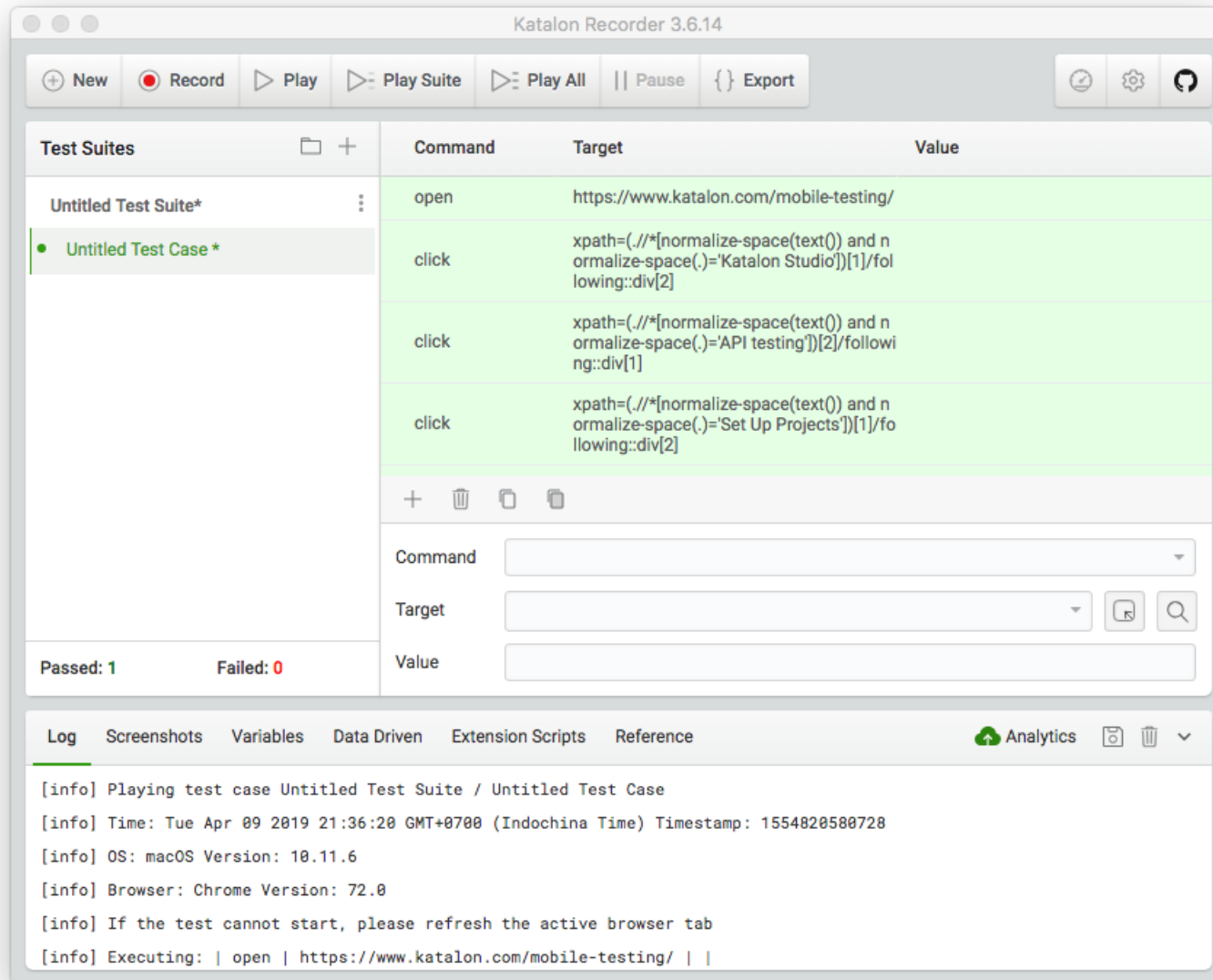
# Record and Playback

- Tools record users' actions performed on AUT
- Often, scripts are generated after recording
- Tools playback what is recorded
- Popular feature in many automation tools

# Selenium's Record and Playback



# Katalon Recorder



# Record and Playback (cont'd)

## ■ Advantages

- ❑ Easy to use
- ❑ No programming skills needed
- ❑ Good for learning

## ■ Problems

- ❑ Scripts can only be created when AUT is ready
- ❑ Does not test anything. Checkpoints are needed to test
- ❑ Only suitable with UI testing
- ❑ Small changes in UI can cause tests to stop
- ❑ Hard to manage and maintain
  - Lots of test scripts are created

## ■ Not a good approach for advanced testing

# Linear Scripting

- Scripts are created to test AUT
- Use some programming languages
- Scripts are also created with Record and Playback
- A test project can have many test suites
  - A test suite has one or more test cases
- Good for simple test cases
- Difficult for large automation

# Linear Scripting (cont'd)

```
# -*- coding: utf-8 -*-
from selenium import selenium
import unittest, time, re

class gtest(unittest.TestCase):
    def setUp(self):
        self.verificationErrors = []
        self.selenium = selenium("localhost", 4444, "chrome", "https://www.google.com/")
        self.selenium.start()

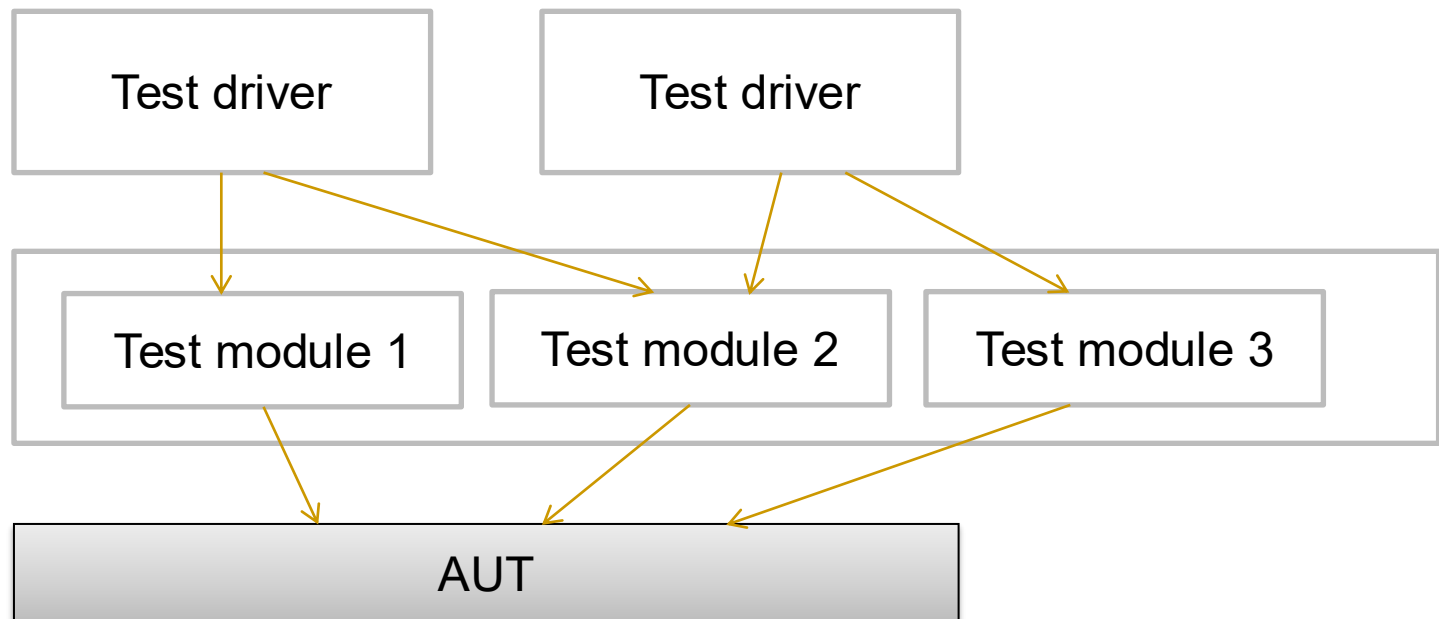
    def test_gtest(self):
        sel = self.selenium
        sel.open("/?gws_rd=ssl")
        sel.click("id=sb_ifc0")
        sel.type("id=lst-ib", "this is a selenium testing")
        sel.click("link=Selenium - Web Browser Automation")
        sel.wait_for_page_to_load("30000")
        sel.click("link=Projects")
        sel.wait_for_page_to_load("30000")
        sel.type("id=q", "what is selenium")
        sel.click("id=submit")
        sel.wait_for_page_to_load("30000")

    def tearDown(self):
        self.selenium.stop()
        self.assertEqual([], self.verificationErrors)
```



# Modular Scripting

- Place test scripts into functions or modules
- Use drivers to call these functions or modules to execute scripts on AUT

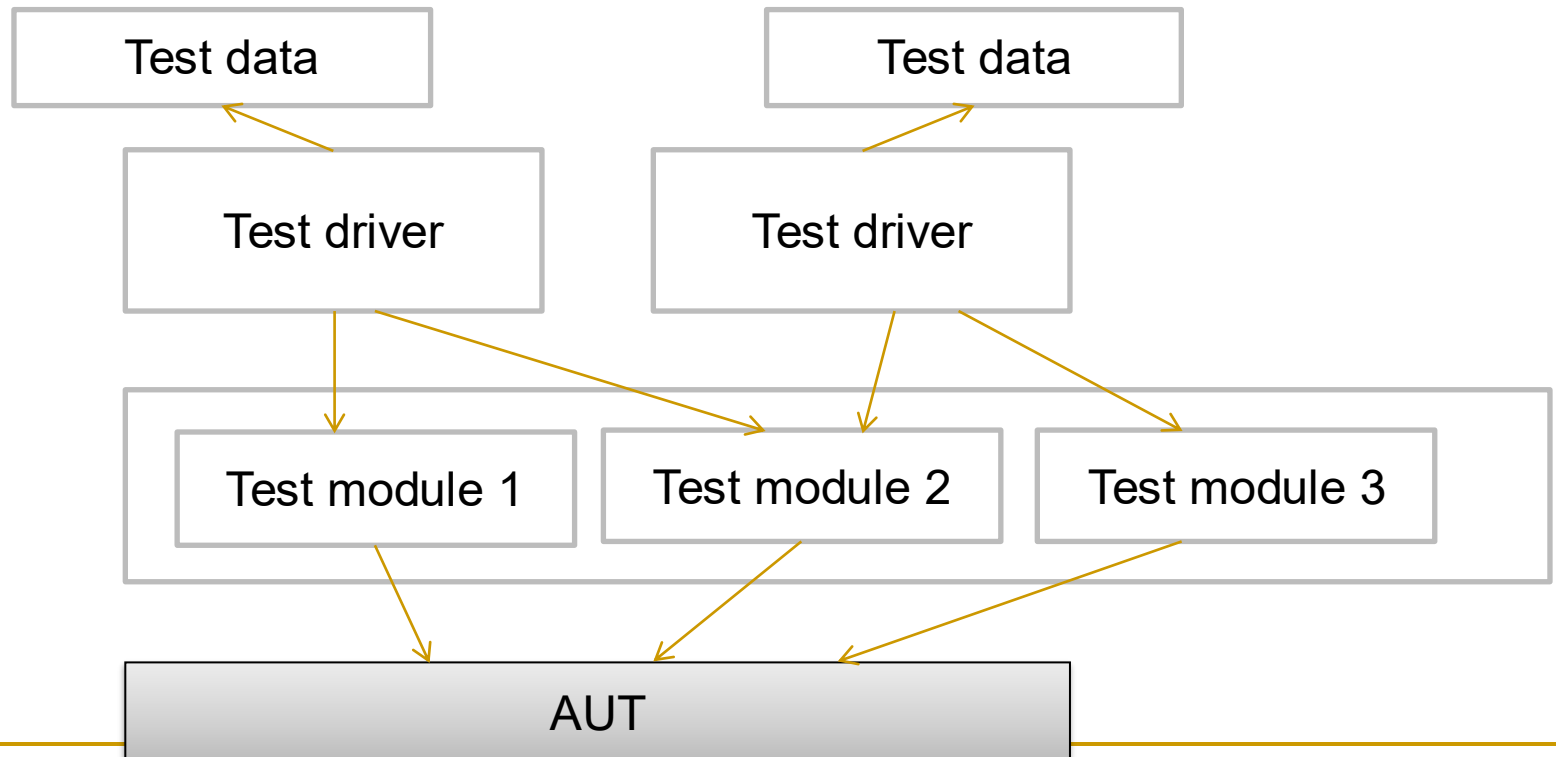


# Modular Scripting (cont'd)

- Advantages: just like procedural programming
  - Reuse of scripts
  - Driver scripts are simple
  - Easy to maintain than linear scripting
- Disadvantages
  - Requiring building libraries/functions initially
  - Test data is embedded directly in scripts
    - Not flexible in changing data
- Requiring programming skills
- Working with simple as well as large tests

# Data-Driven Scripting/Framework

- Data is separated from test scripts
- Test scripts read data automatically from files
- Can test multiple data items, different inputs



# Data-Driven Scripting (cont'd)

- Test driver scripts can use multiple data items
  - E.g., testing different username and password combinations
  - Flexible in changing inputs
- Programming skills not needed when updating data
- Good for separating roles
  - Developers responsible for scripting
  - Testers responsible for test data

# Data-Driven Scripting (cont'd)

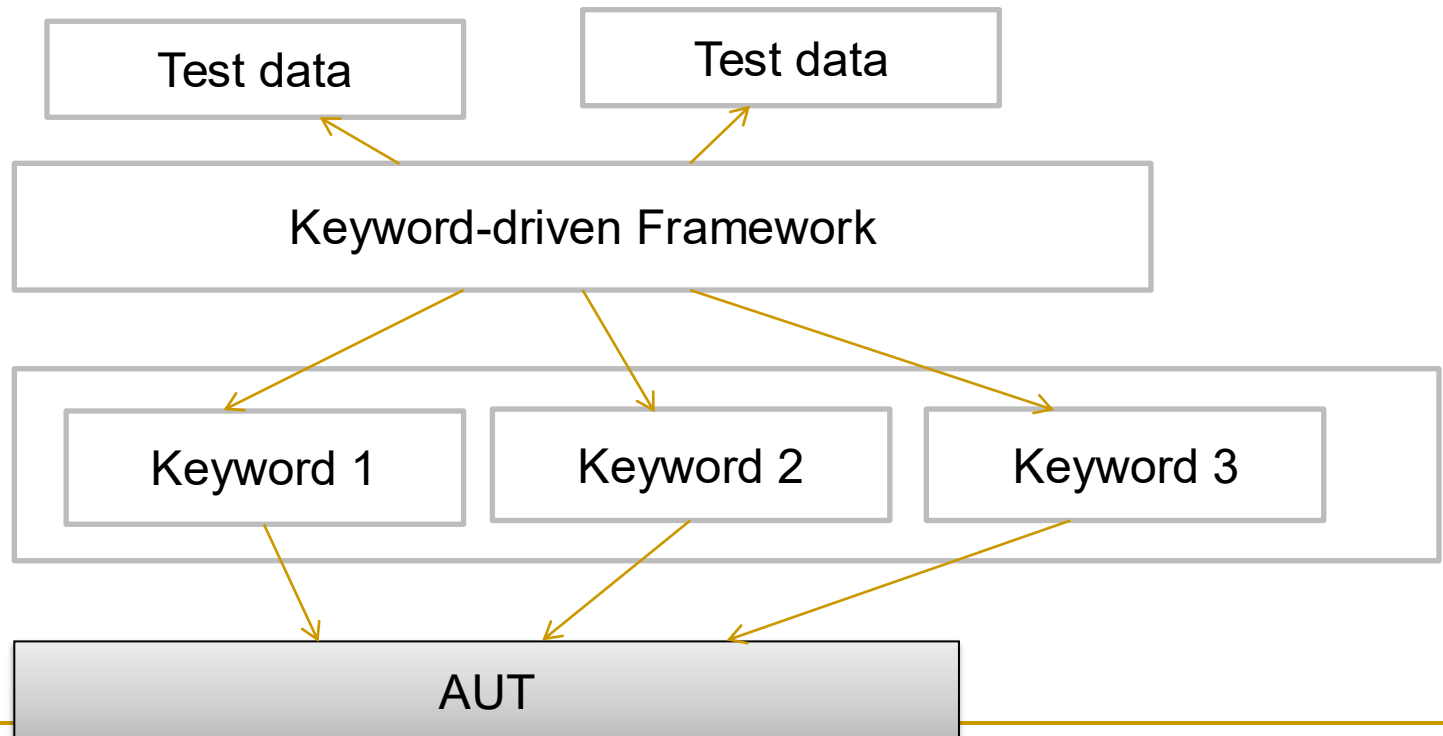
- Disadvantages

- Initial effort is required to create framework (parser, libraries, etc)
  - Just like building a new tool
- New tests need new driver scripts
- Require programming skills to create new driver scripts

- Good solution for large scale automation

# Keyword-Driven Testing

- Separate test scripts from
  - test data
  - directives (keywords) on how to use data
- Keywords and data drive test execution



# Keyword-Driven Testing (cont'd)

|    | A             | B              | C                                    | D |
|----|---------------|----------------|--------------------------------------|---|
| 1  | TestCase Name | Keyword        | Description                          |   |
| 2  | TC_01         | gmLogin        | Login to Gmail                       |   |
| 3  |               | gmEmails       | Find number of emails received today |   |
| 4  |               | gmLogout       | Logout from Gmail                    |   |
| 5  | TC_02         | gmLogin        | Login to Gmail                       |   |
| 6  |               | gmSend         | Send Email                           |   |
| 7  |               | gmLogout       | Logout from Gmail                    |   |
| 8  | TC_03         | fbLogin        | Login to Facebook                    |   |
| 9  |               | fbNotification | Find unread notifications            |   |
| 10 |               | fbLogout       | Logout from Facebook                 |   |
| 11 |               |                |                                      |   |

www.automationrepositrory.com

Each keyword associated with the test case is mentioned.

For ease of understanding, proper description is provided with each keyword

# Keyword-Driven Testing (cont'd)

## ■ Advantages

- ❑ All tests can be handled by one framework
- ❑ Tests can be created from keywords
- ❑ Non-programmers can create tests
- ❑ Separate test data and scripts

## ■ Disadvantages

- ❑ Effort required to build framework upfront
- ❑ Programming skills needed to build framework

## ■ Good solution supported by many commercial tools



# Combination of the approaches

- Advanced tools today allow to combine multiple approaches
- Tools integrate recorder, scripts, keywords, data
- Tools
  - Katalon Studio
  - TestComplete
  - UFT
  - Ranorex

# Topics covered

- Overview of test automation
- Typical test automation process
- Testing approaches
- ➔ ■ Automation tools

# Test Automation Tools

- Hundreds of test automation tools available in market
- Commercial tools
  - ❑ Usually powerful
  - ❑ Well supported
  - ❑ Rich feature set
  - ❑ Some are expensive, unluckily
- Open source
  - ❑ Free, of course
  - ❑ Many are good to use, others not that good
  - ❑ Support uncertain

# Test Automation Tools (cont'd)

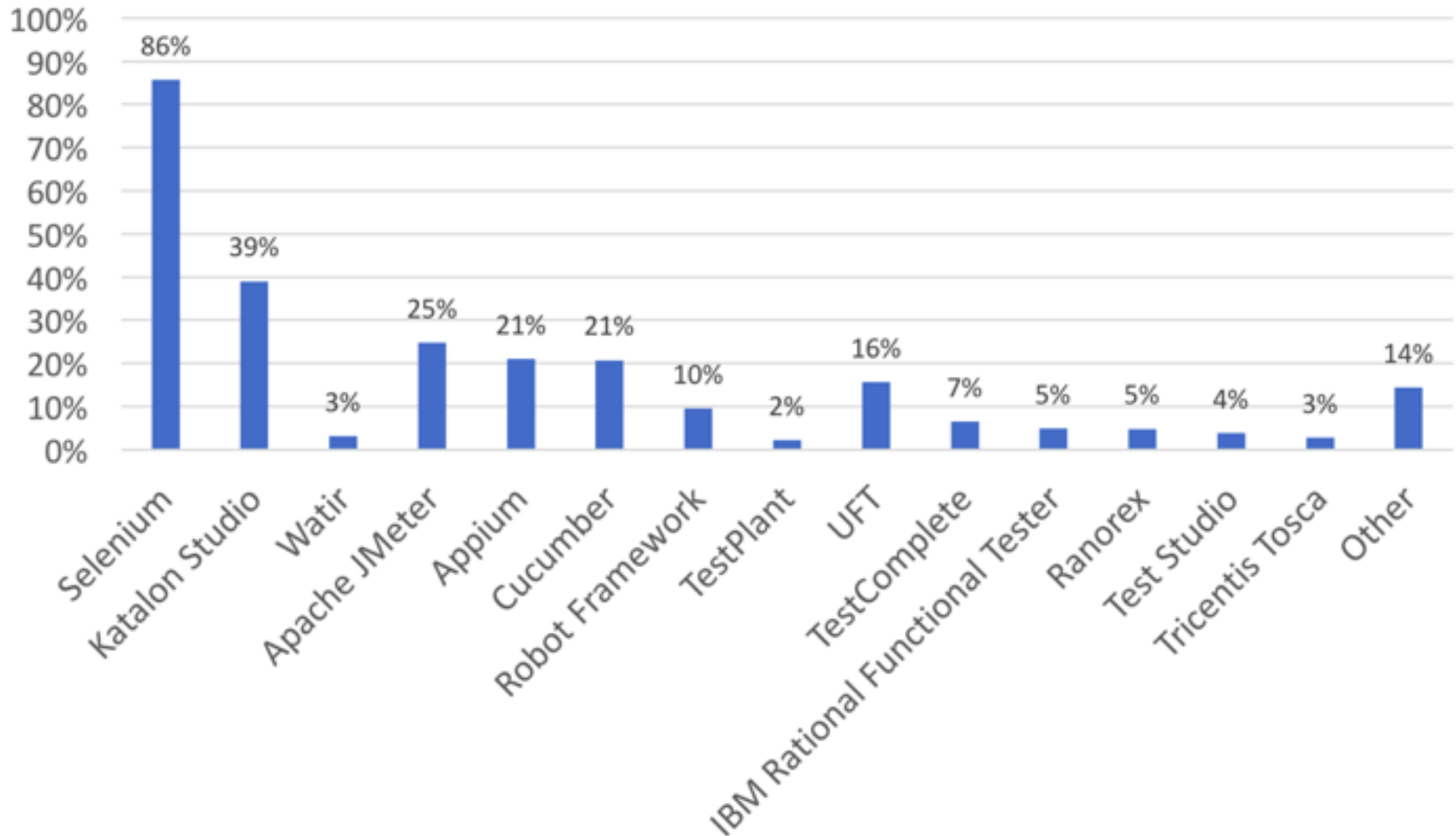
## ■ Popular commercial tools

- ❑ Katalon
- ❑ Postman
- ❑ OpenText UTF One
- ❑ TestComplete
- ❑ Ranorex
- ❑ eggPlant
- ❑ Tricentis Tosca

## ■ Popular open source and/or free tools

- ❑ Selenium
- ❑ Playwright
- ❑ Cucumber
- ❑ Cypress
- ❑ Appium
- ❑ Robot Framework

# Most popular tools



Source: “The most striking problems in test automation: A survey”, 2018. Katalon.com

# Want to Work in Test Automation?

- Programming skills are a plus
  - Scripting languages: Python, Ruby, Java, C#, etc.
- Regular expressions
- SQL
- Learning to use test automation tools
  - Selenium
  - Katalon Studio

# Conclusions

- Test automation is now a trend in software development
  - Customers require to apply test automation
  - Demand is high
- Test automation is essential for DevOps
- It does not replace manual testing

# Testing and evaluation review

- Report approaches you will pursue to evaluate and test your systems